Project: Implementing microservices with Code Pipeline and ECS

Course-end Project 2

Description

Objective

The objective of this project is to showcase the end-to-end process of deploying a microservices-based application with separate Backend (NodeJS) and Frontend (React) services using AWS CodePipeline and Amazon ECS. This project demonstrates the automation of CI/CD workflows, efficient containerization of both backend and frontend applications, and the deployment of these services on a scalable ECS cluster. The setup ensures high availability, seamless updates, and a reliable architecture utilizing serverless containers with AWS Fargate.

Problem Statement and Motivation

Real-time scenario:

DevBank, a rapidly expanding financial technology company, offers a digital banking platform to clients across North America. To enhance scalability and handle increasing user transactions, DevBank is transitioning its backend services (NodeJS API) and frontend interface (React application) to a microservices architecture deployed on AWS.

As a DevOps Engineer at DevBank, you are tasked with implementing automated CI/CD pipelines for both backend and frontend services. This project involves using Docker for containerization, AWS CodePipeline for automating builds, and Amazon ECS with AWS Fargate for deploying the services in a serverless, high-availability setup. Your responsibility is to ensure an efficient, automated deployment process that reduces release time and maintains a seamless user experience, enabling DevBank to scale its platform and deliver reliable services to its growing customer base.

Repositories

- [ReactFrontend]
- [NodeBackend]

Frontend buildspec.yml

```
version: 0.2

phases:
  pre_build:
    commands:
      - echo Logging in to DockerHub...
      - echo $DOCKERHUB_PASS | docker login -u $DOCKERHUB_USER --password-stdin
      - REPOSITORY_URI="docker.io/pradeepviswa/capstonefrontend"
      - CONTAINER_NAME="frontend-container"
      - IMAGE_TAG="latest"
  build:
    commands:
      - echo Build started on `date`
      - echo Building the Docker image...
      - docker build -t $REPOSITORY_URI:$IMAGE_TAG .
  post_build:
    commands:
      - echo Build completed on `date`
      - docker push $REPOSITORY_URI:$IMAGE_TAG
      - echo Writing image definitions file...
      - printf '[{"name":"'$CONTAINER_NAME'","imageUri":"%s"}]' $REPOSITORY_URI:$IMAGE_TAG
> imagedefinition.json
artifacts:
    files: imagedefinition.json
```

Backend buildspec.yml

```
version: 0.2

phases:
  pre_build:
    commands:
      - echo Logging in to DockerHub...
      - echo $DOCKERHUB_PASS | docker login -u $DOCKERHUB_USER --password-stdin
      - REPOSITORY_URI="docker.io/pradeepviswa/capstonebackend"
      - CONTAINER_NAME="backend-container"
      - IMAGE_TAG="latest"
  build:
    commands:
      - echo Build started on `date`
      - echo Building the Docker image...
      - docker build -t $REPOSITORY_URI:$IMAGE_TAG .
  post_build:
    commands:
      - echo Build completed on `date`
      - docker push $REPOSITORY_URI:$IMAGE_TAG
      - echo Writing image definitions file...
      - printf '[{"name":"'$CONTAINER_NAME'","imageUri":"%s"}]' $REPOSITORY_URI:$IMAGE_TAG
> imagedefinition.json
artifacts:
    files: imagedefinition.json
```

AWS Login

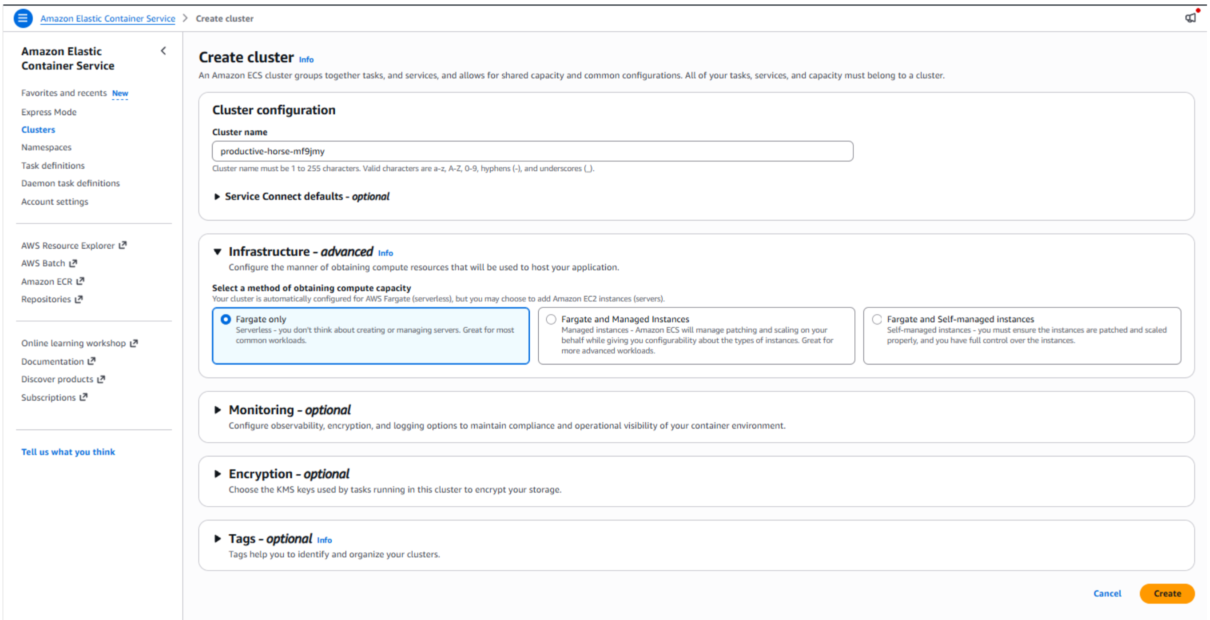
Region: us-east-1

AWS VPC

ECS Cluster

Name: vibrant-lion-otuntq (use the default generated name)

Infrastructure: Fargate Only (Serverless)

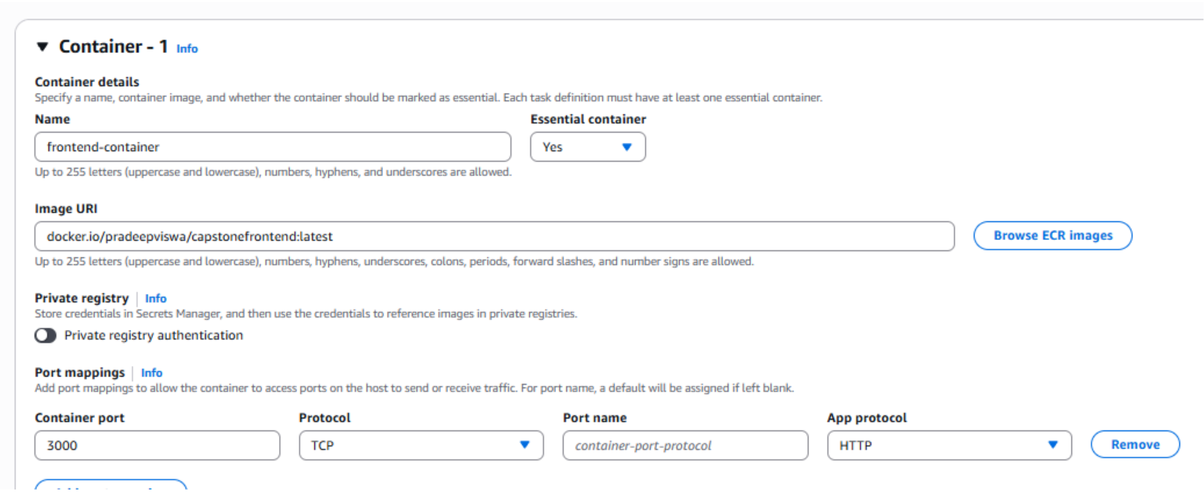


Create Task Definitions

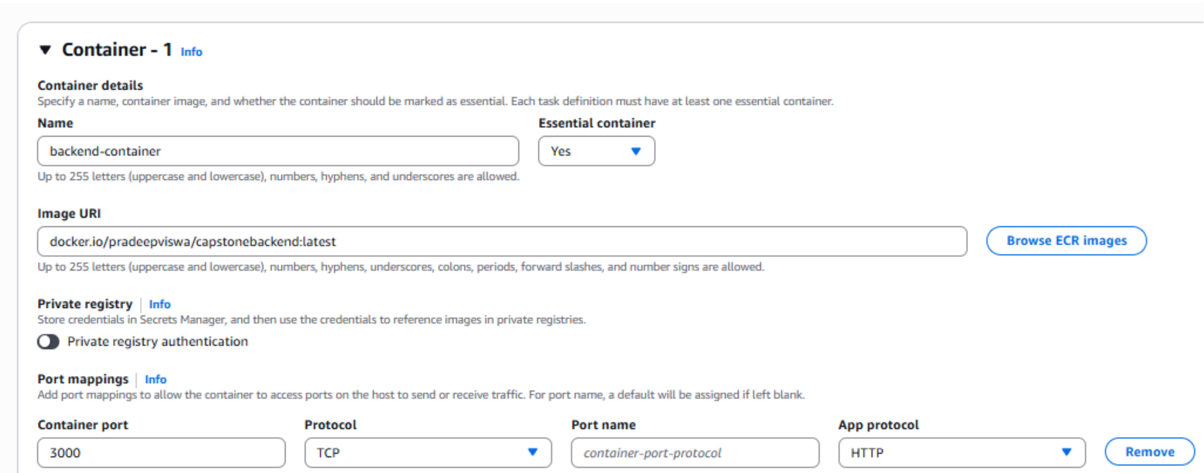
Amazon Elastic Container Service > **Task definitions**

Create task definition for both frontend and backend:

1. container-port should be set to 3000
2. Container Detail:
    - For Frontend:
        1. Name: frontend-container
        2. URI: docker.io/pradeepviswa/capstonefrontend:latest

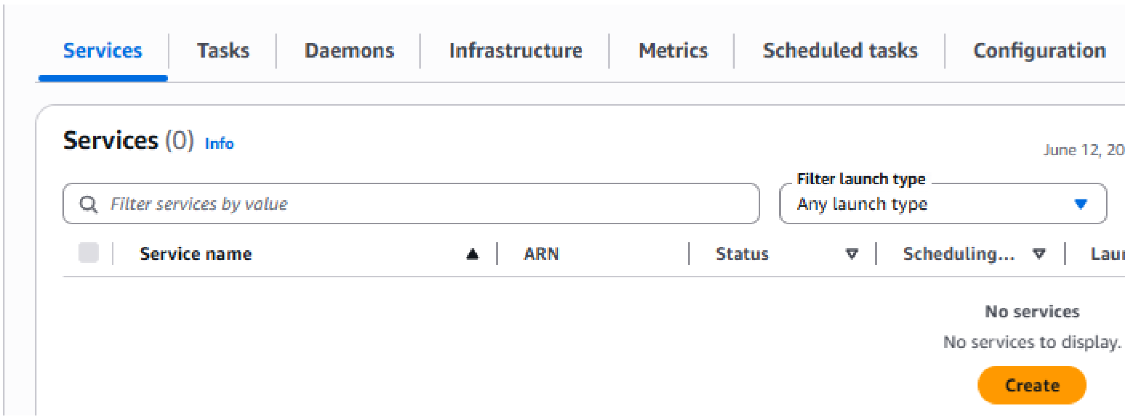


    - For Backend:
        1. Name: backend-container
        2. URI: docker.io/pradeepviswa/capstonebackend:latest



Create ECS Service

Go to cluster (**vibrant-lion-otuntq)** > service

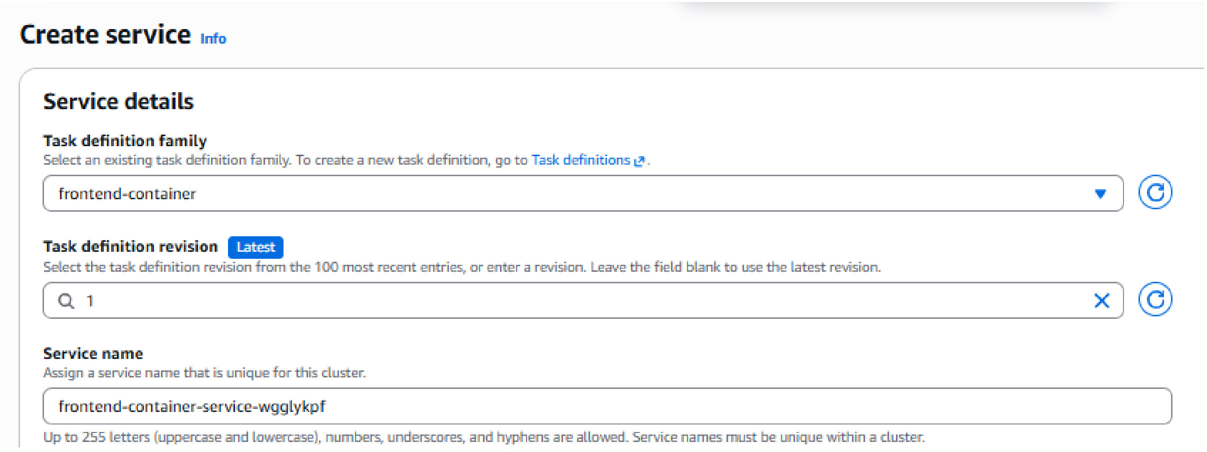


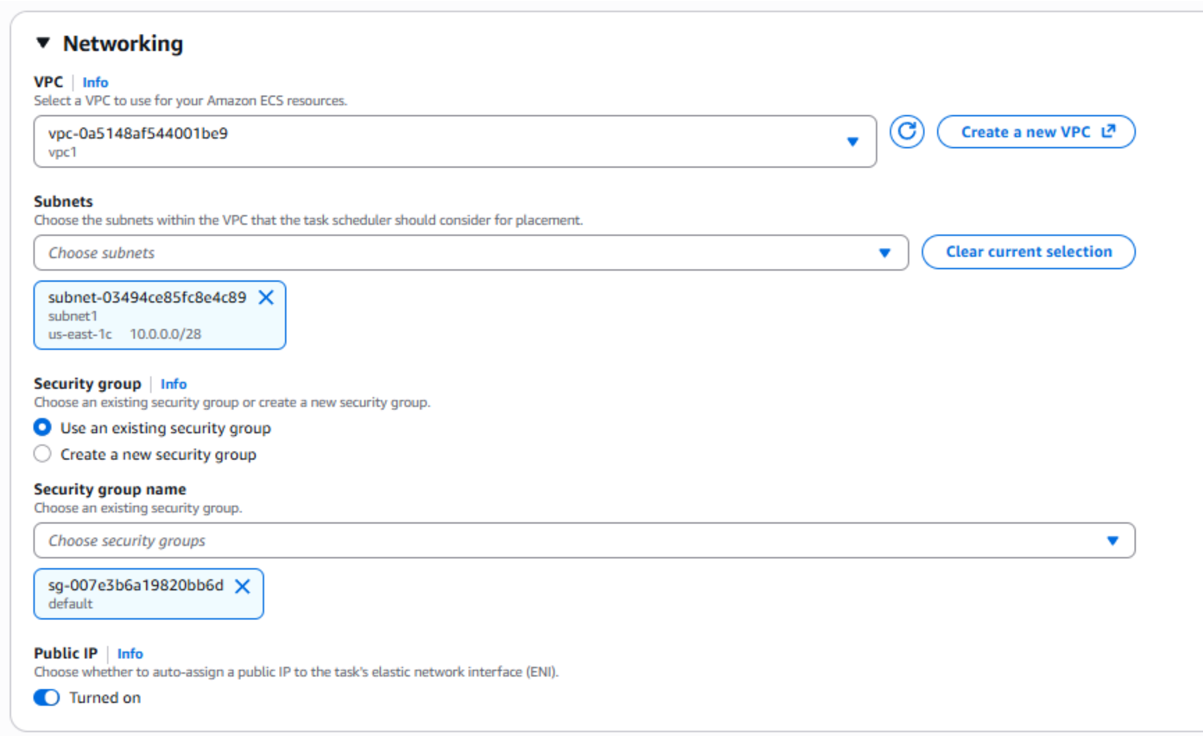
Create Service for both frontend and backend:

1. choose the task definition Family
2. Networking
    - Choose the previously created VPC
    - Choose all the public subnet
    - create new security group
    - inbound rule
        - Type = Custom TCP
        - Port range = 3000
        - Source = Anywhere

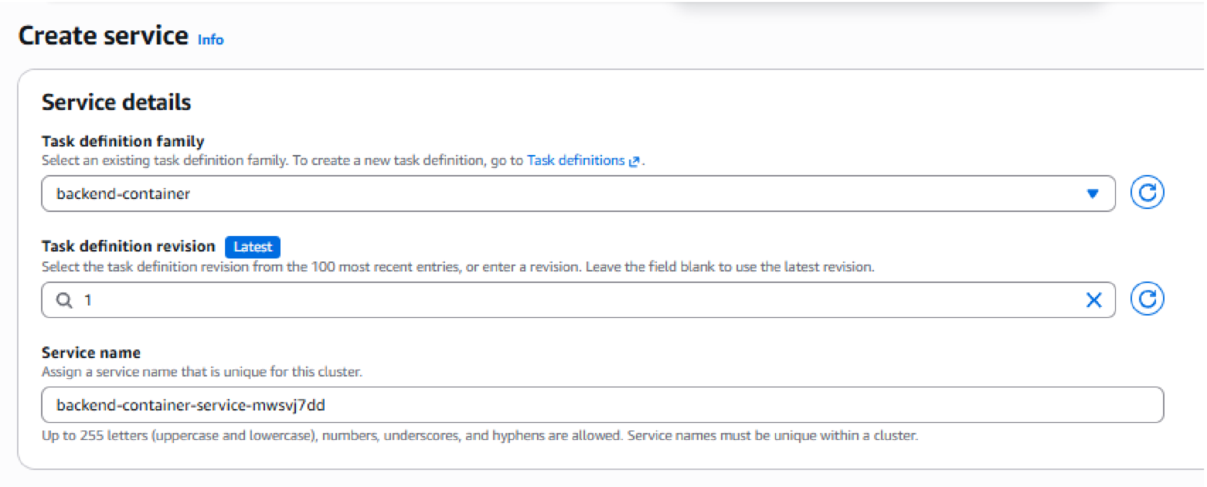
a. Note: you need to create the Security group only once, when you create the first service. Reuse the group in second service

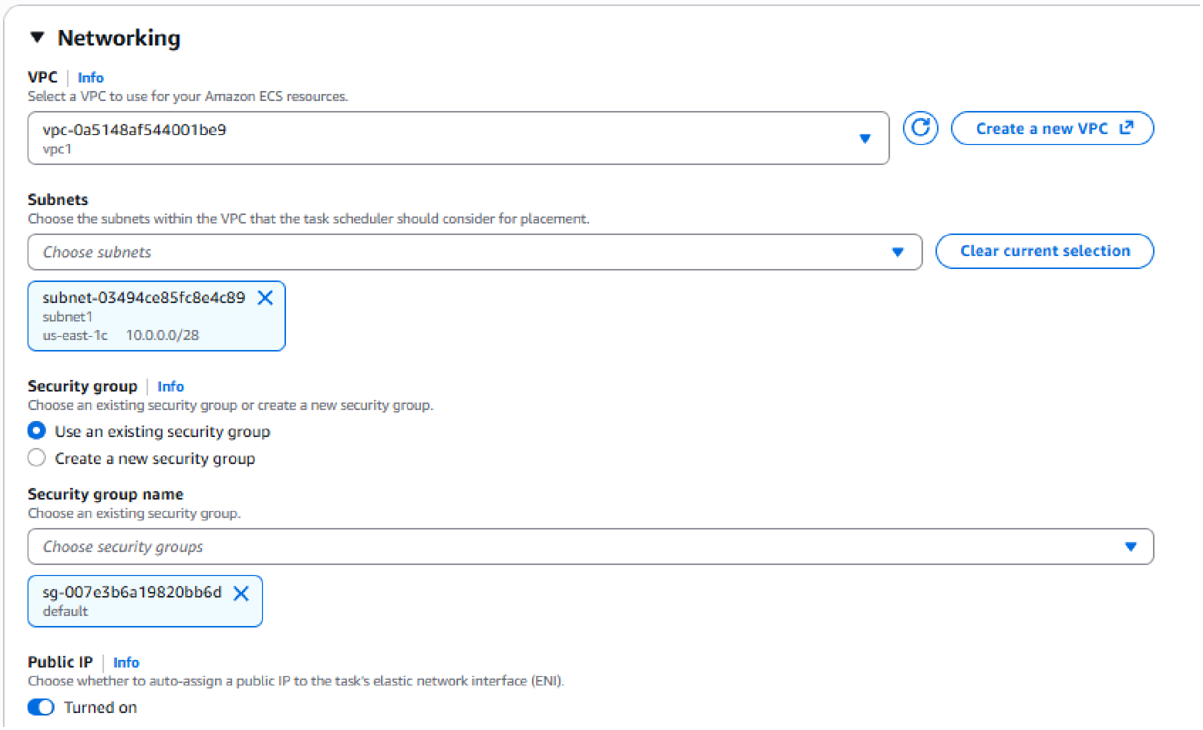
Frontend service



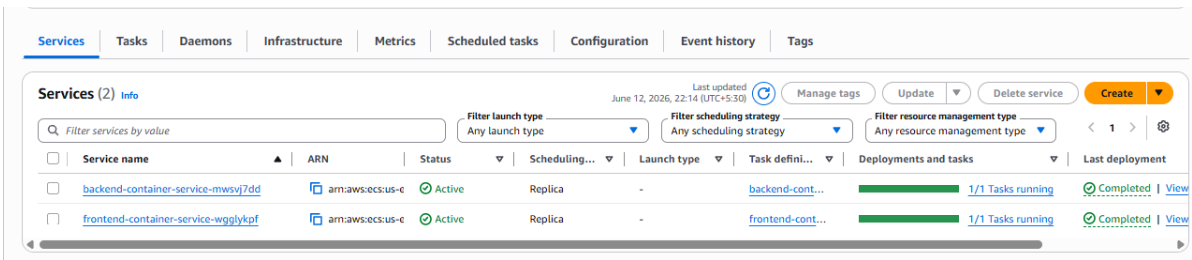


Backend Service





Service list



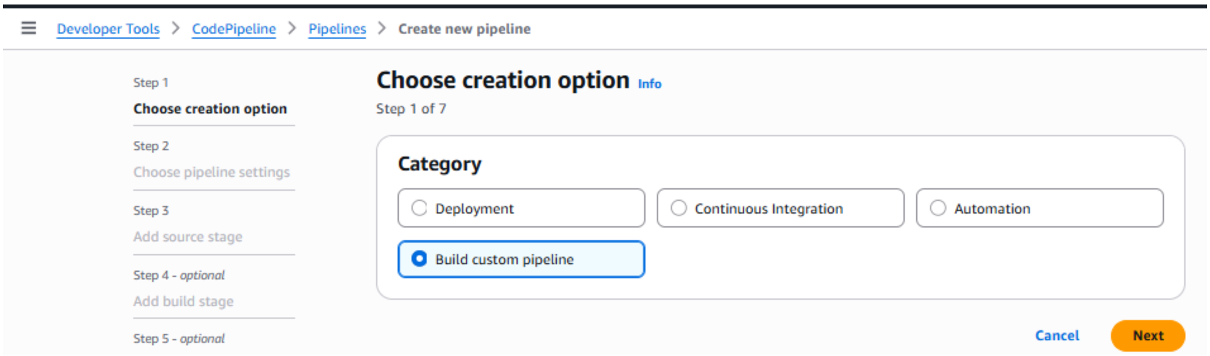
Setup CodePipeline

Create two pipelines : FrontendPipeline and BackendPipeline

Source stage

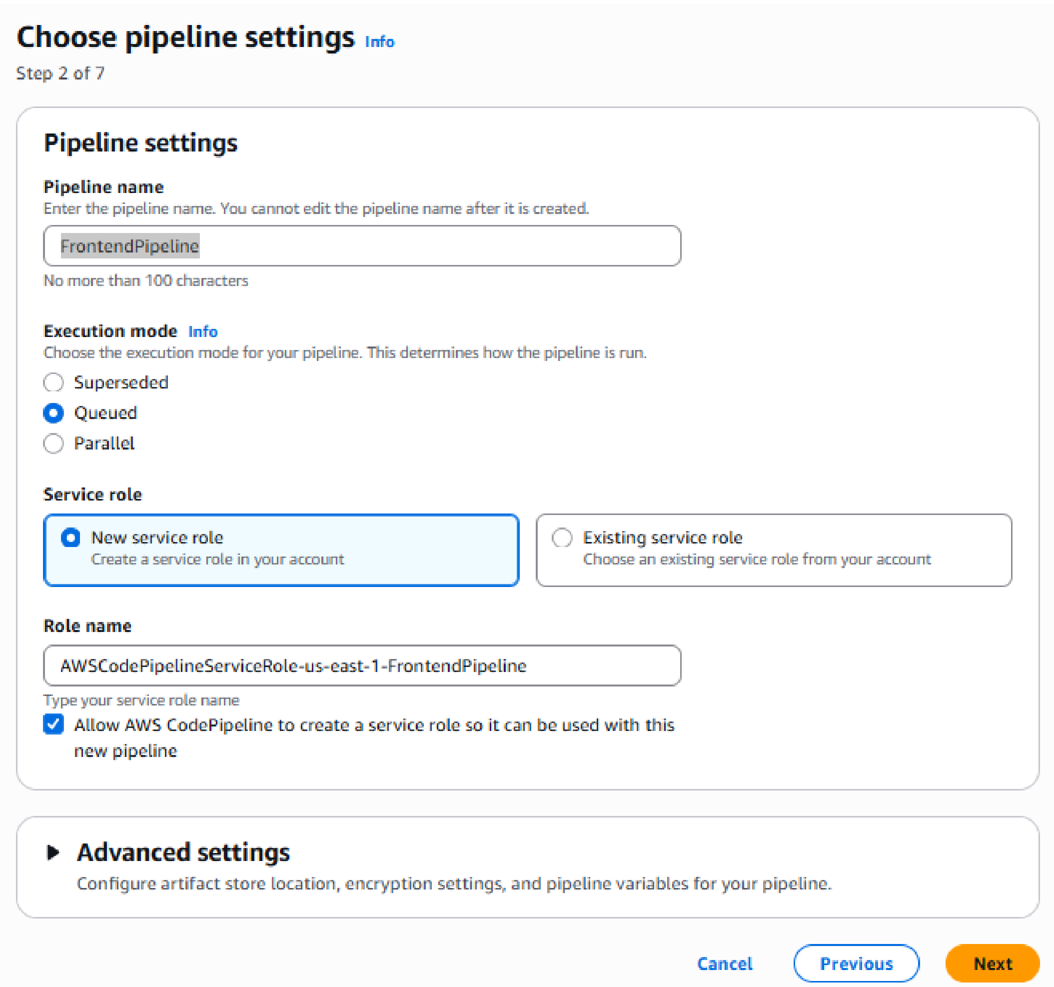
Create Code pipeline for Backend

Developer Tools > CodePipeline > Pipelines > Create new pipeline > build custom pipeline



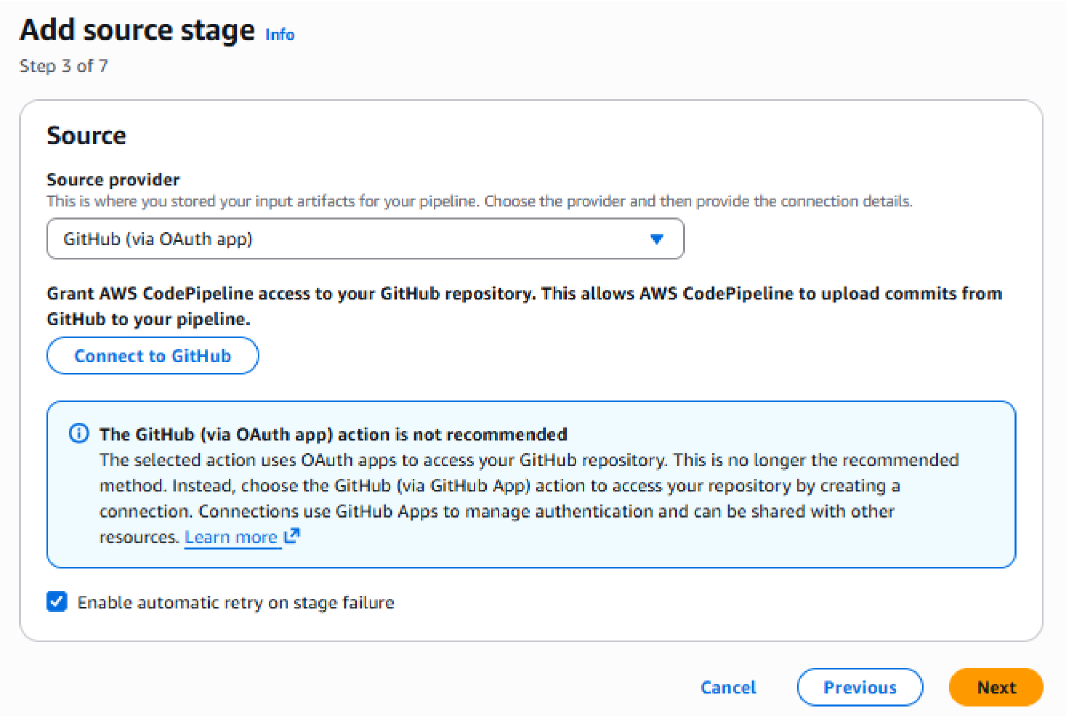
Pipeline settings

Name: FrontendPipeline



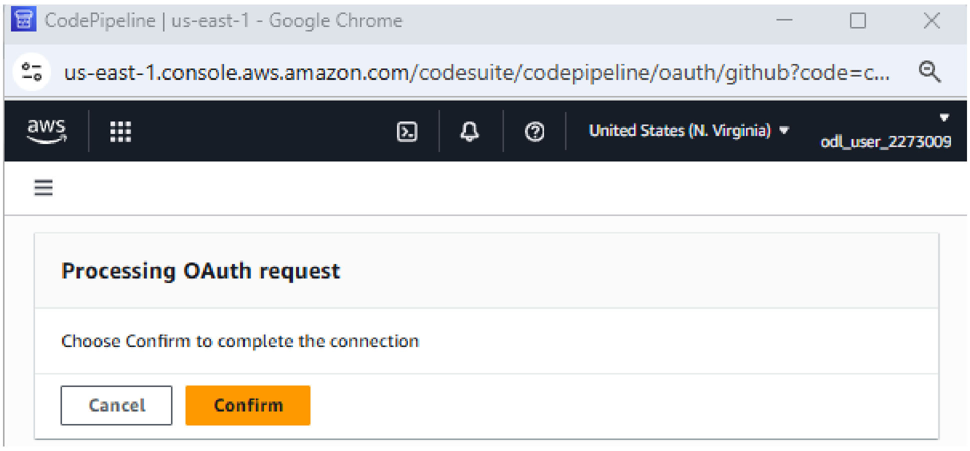
Source

Github (via OAuth app) > connect to Github

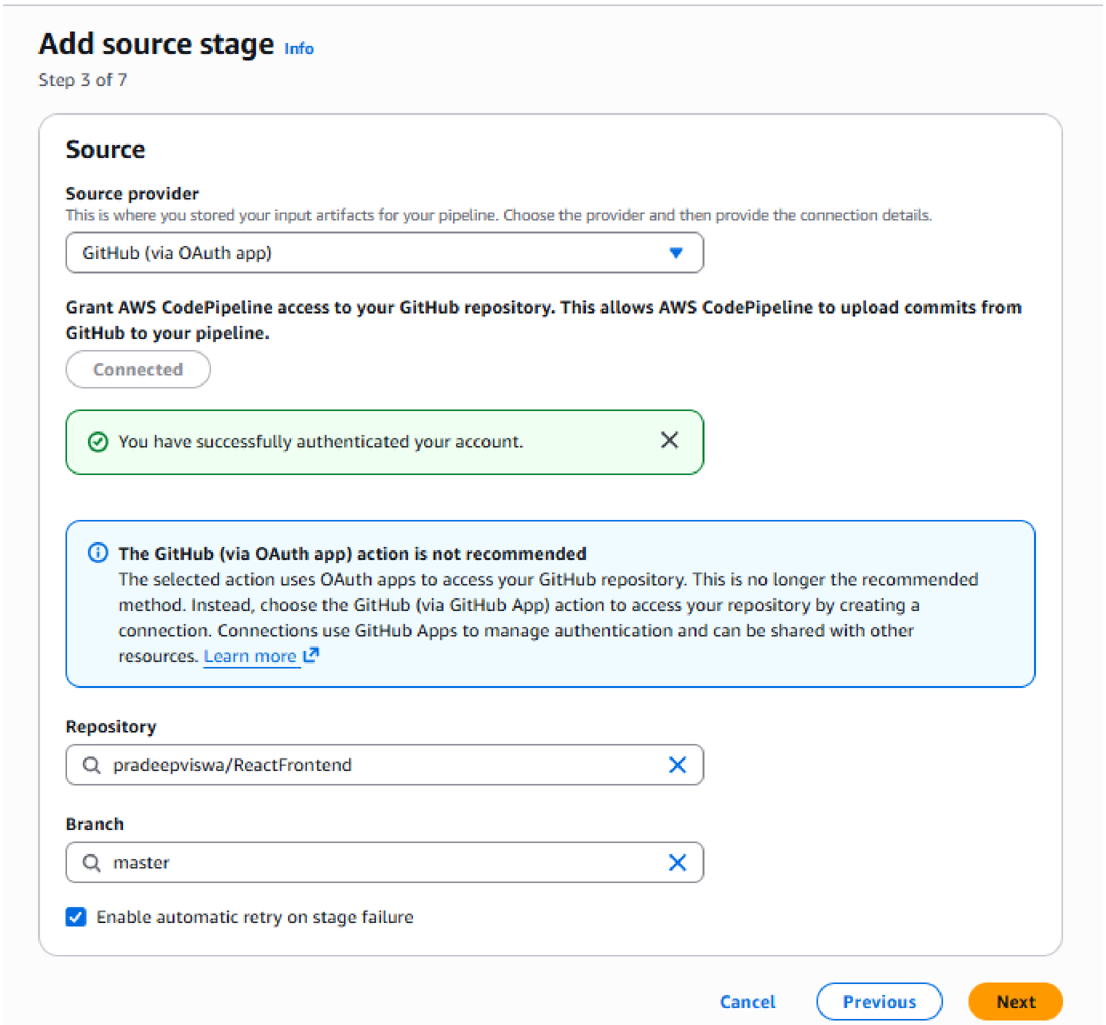


Connect with github

Authenticate



Select repository and branch

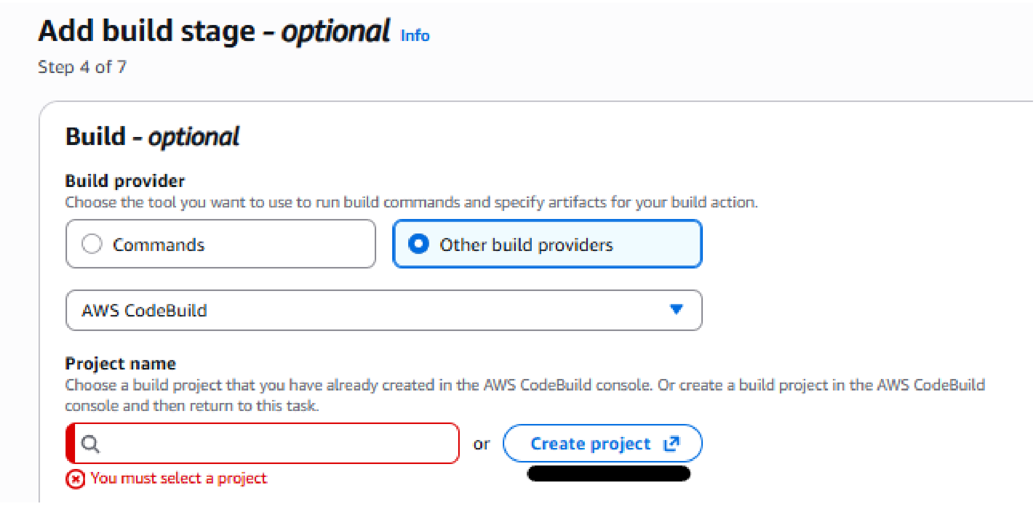


Build

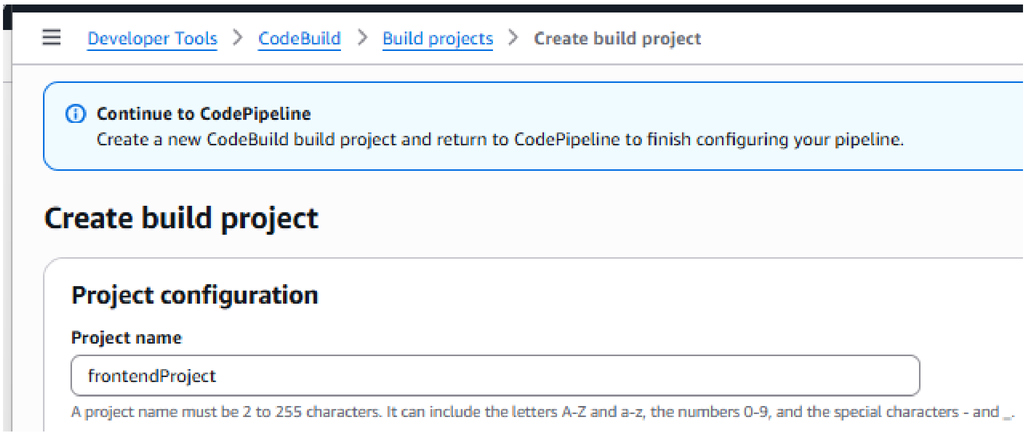
Build provider: Other build provider

AWS CodeBuild

Create Project



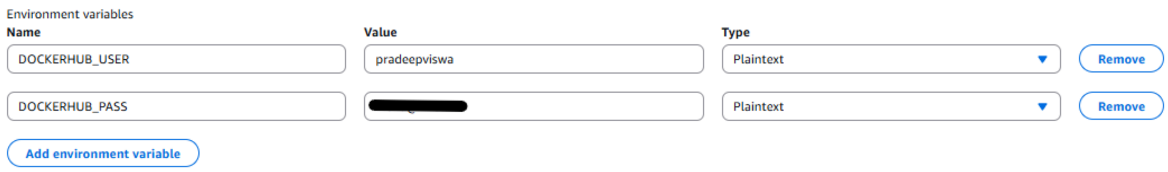
Project name: frontendProject



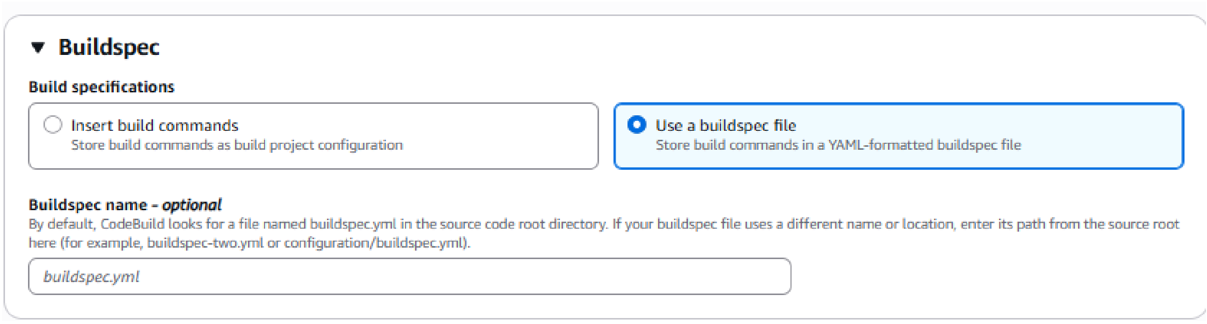
add two environment variables under “Additional configuration”:

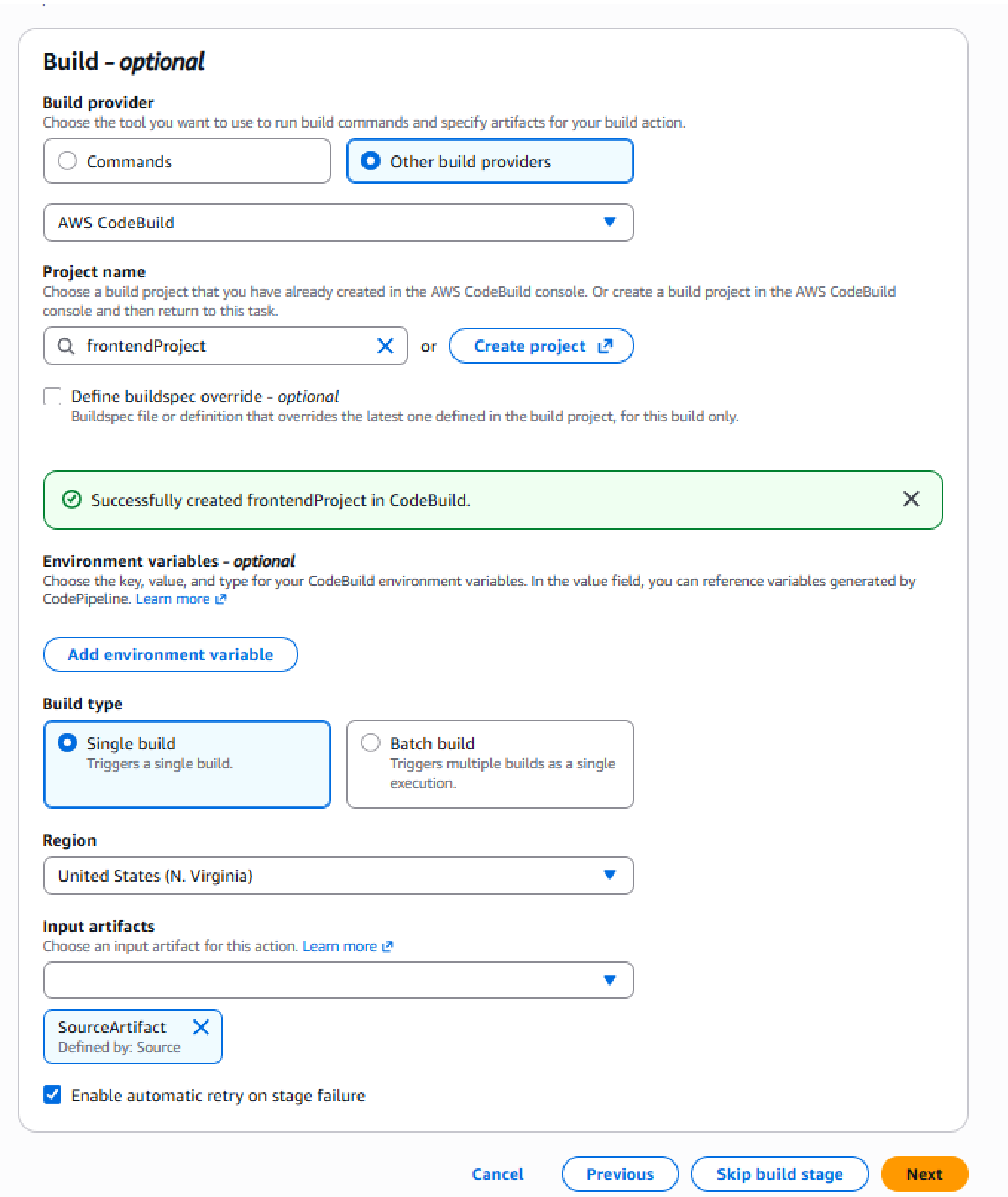
i. “DOCKERHUB_USER”value = *your dockerhub user* type = plaintext

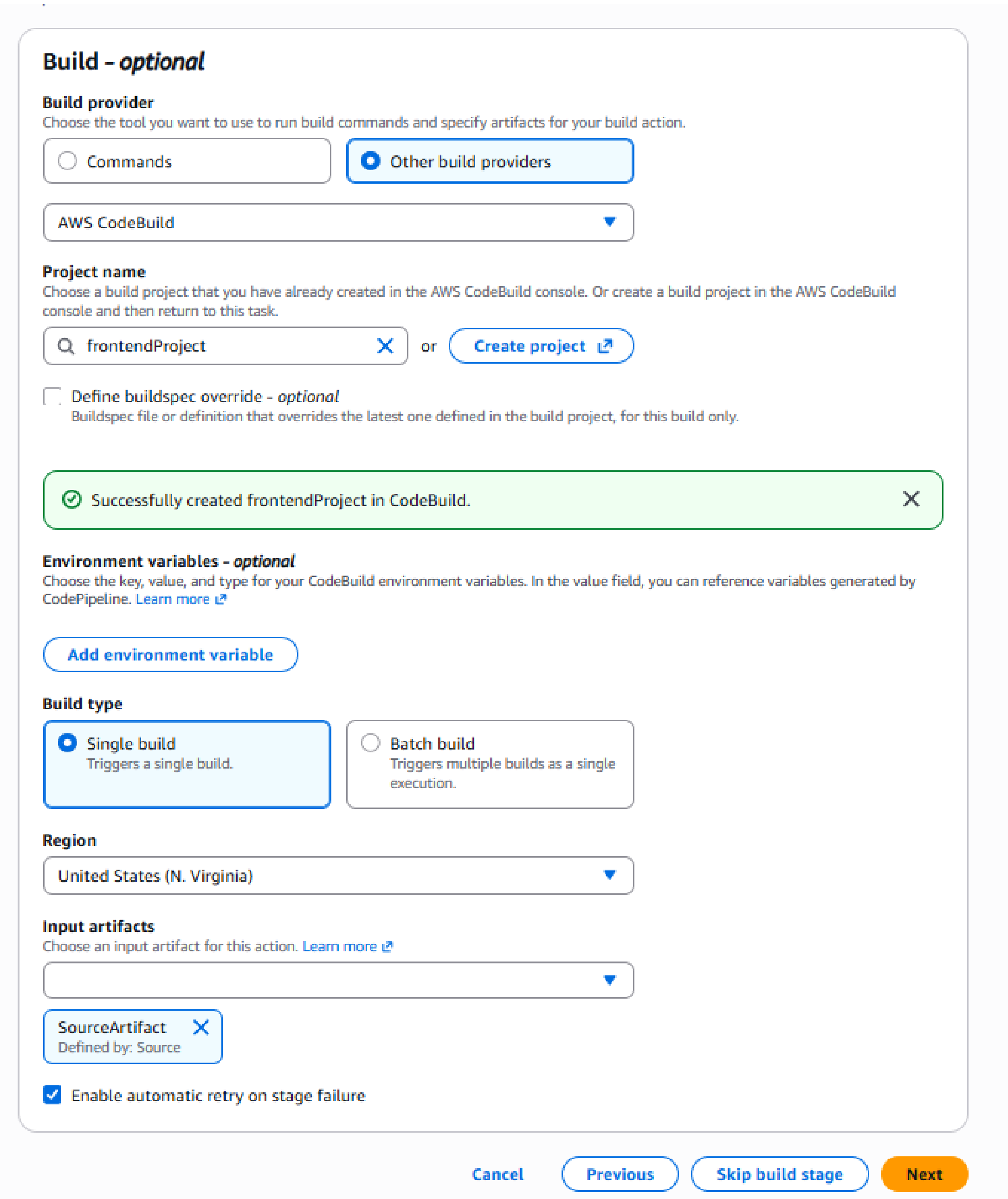
ii. “DOCKERHUB_PASS” value = *your dockerhub password* type = plaintext



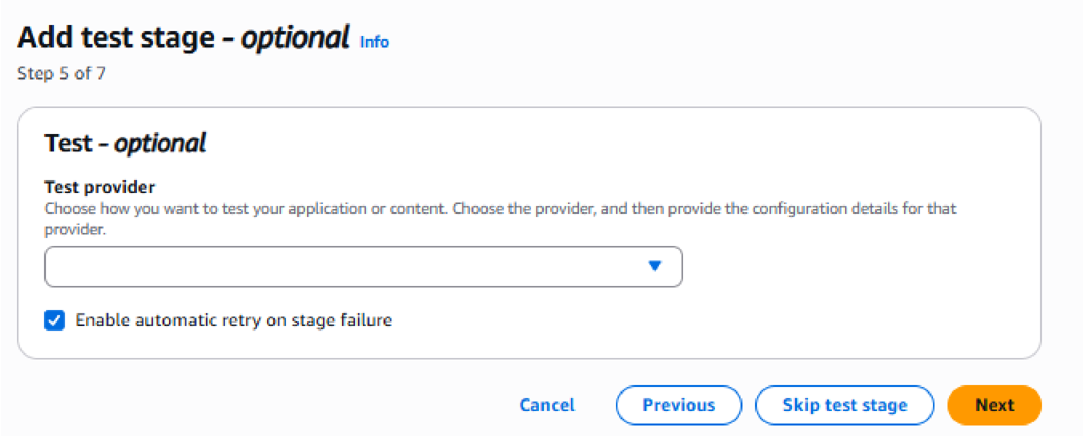
Buildspec: Use a buildspec file




Build - optional
Build provider
Choose the tool you want to use to run build commands and specify artifacts for your build action.
Commands
Other build providers
AWS CodeBuild
Project name
Choose a build project that you have already created in the AWS CodeBuild console. Or create a build project in the AWS CodeBuild console and then return to this task.
frontendProject
or
Create project
Define buildspec override - optional
Buildspec file or definition that overrides the latest one defined in the build project, for this build only.
Successfully created frontendProject in CodeBuild.
Environment variables - optional
Choose the key, value, and type for your CodeBuild environment variables. In the value field, you can reference variables generated by CodePipeline. Learn more
Add environment variable
Build type
Single build
Triggers a single build.
Batch build
Triggers multiple builds as a single execution.
Region
United States (N. Virginia)
Input artifacts
Choose an input artifact for this action. Learn more
SourceArtifact
Defined by: Source
Enable automatic retry on stage failure
Cancel
Previous
Skip build stage
Next


Build - optional
Build provider
Choose the tool you want to use to run build commands and specify artifacts for your build action.
Commands
Other build providers
AWS CodeBuild
Project name
Choose a build project that you have already created in the AWS CodeBuild console. Or create a build project in the AWS CodeBuild console and then return to this task.
frontendProject
or
Create project
Define buildspec override - optional
Buildspec file or definition that overrides the latest one defined in the build project, for this build only.
Successfully created frontendProject in CodeBuild.
Environment variables - optional
Choose the key, value, and type for your CodeBuild environment variables. In the value field, you can reference variables generated by CodePipeline. Learn more
Add environment variable
Build type
Single build
Triggers a single build.
Batch build
Triggers multiple builds as a single execution.
Region
United States (N. Virginia)
Input artifacts
Choose an input artifact for this action. Learn more
SourceArtifact
Defined by: Source
Enable automatic retry on stage failure
Cancel
Previous
Skip build stage
Next

Test Stage : *Skip*



Deploy stage

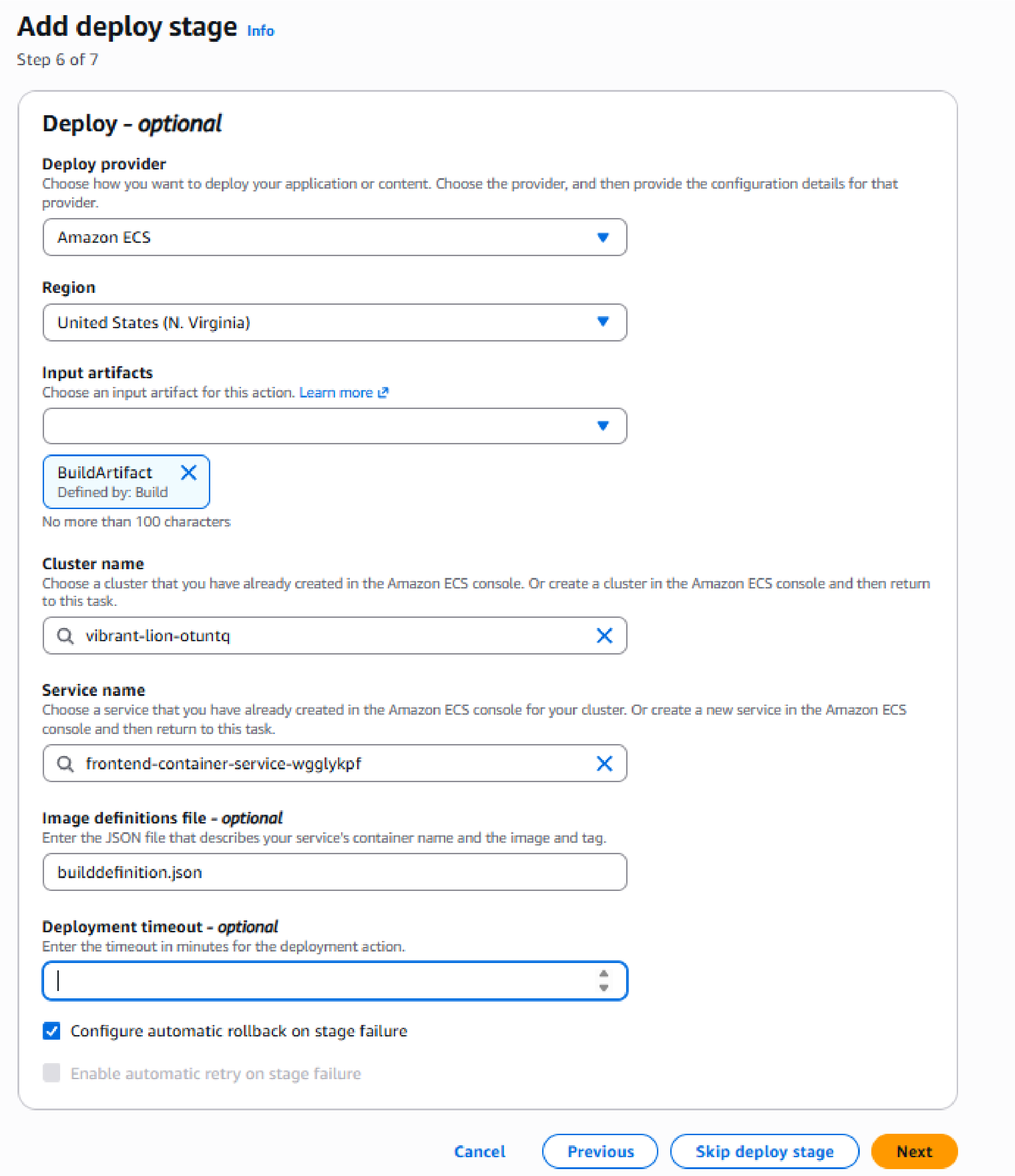
Deploy provider: Amazon ECS

Input artifacts: BuildArtifact

Cluster name: ECS cluster name

Service name: frontend-service

Image definitions file : “builddefinition.json” (for frontend pipeline)



Build pipeline

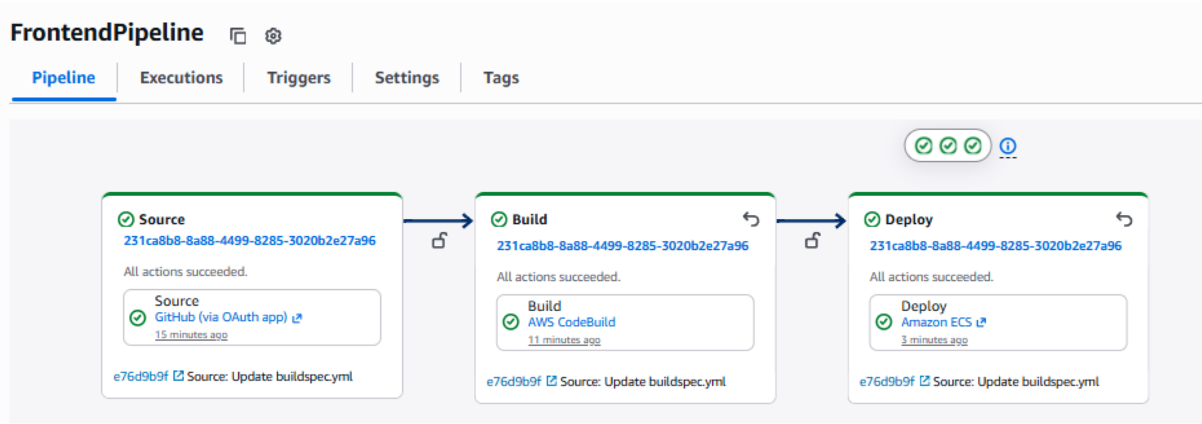
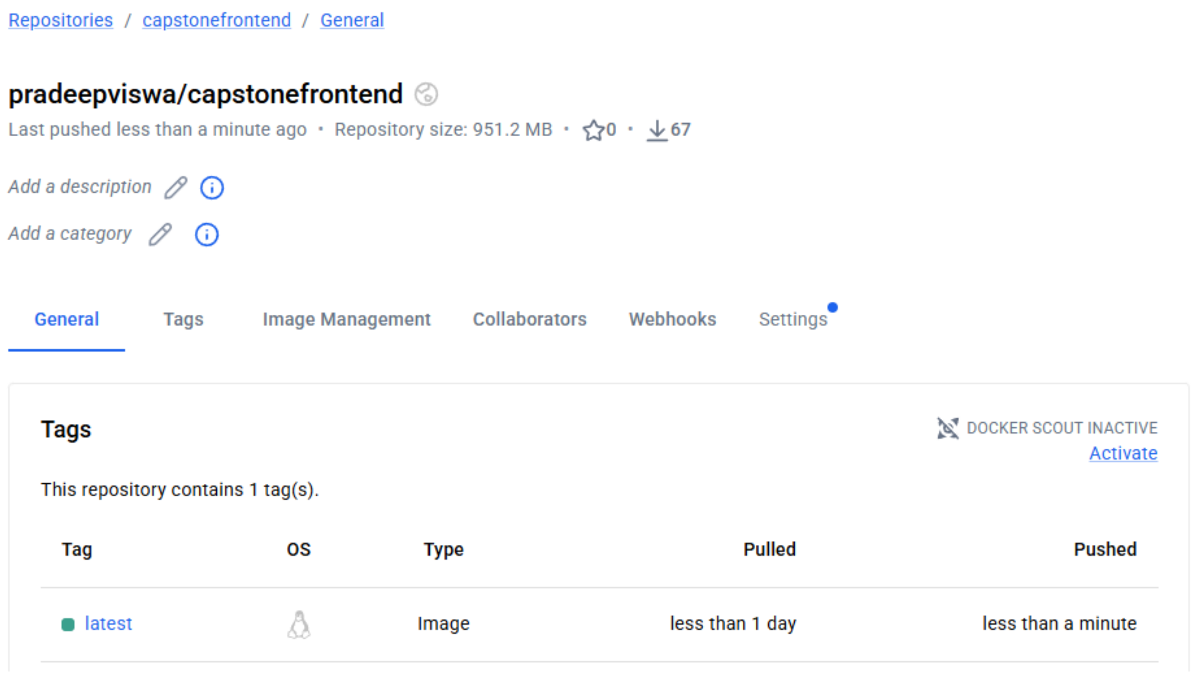
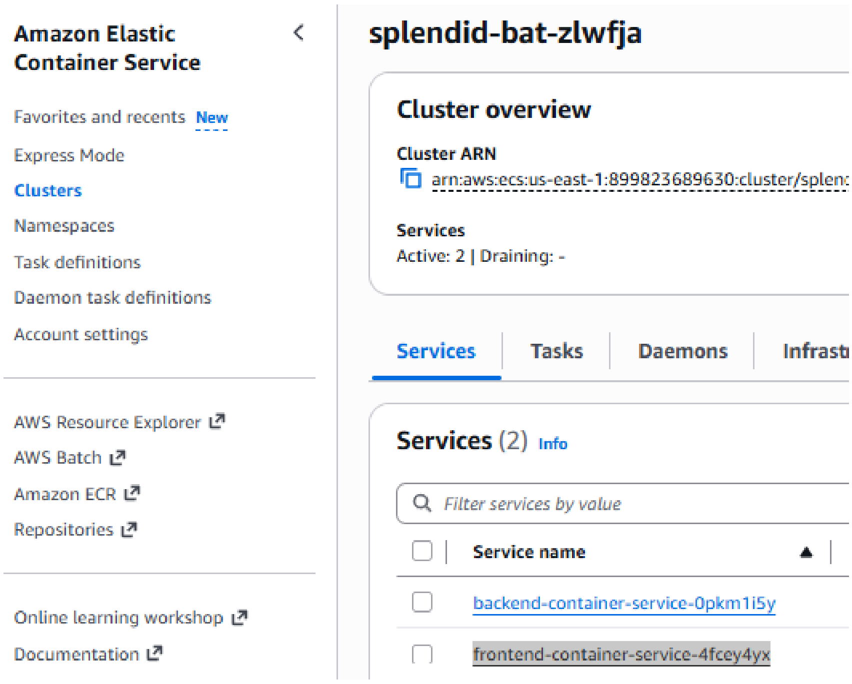


Image has been uploaded to docker-hub

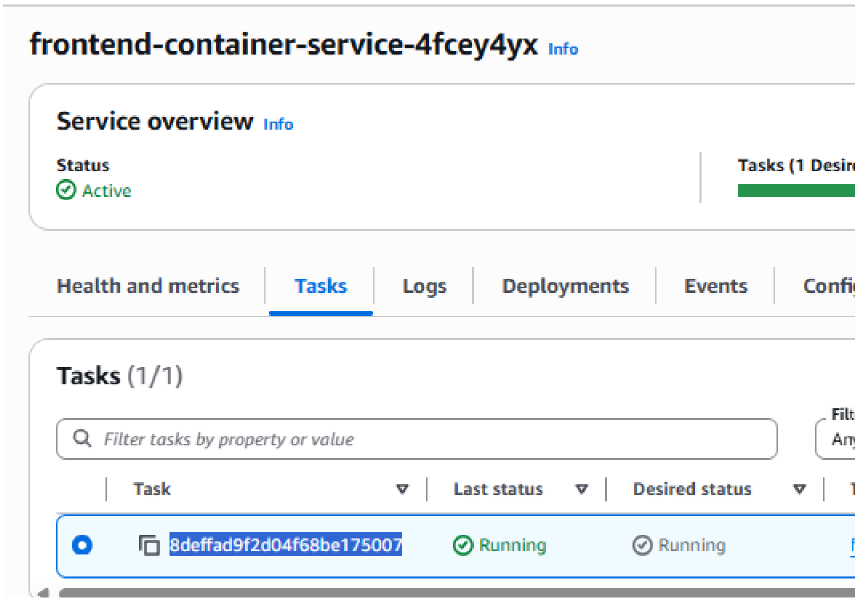


Web page

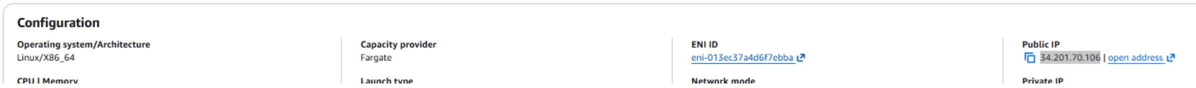
Go to cluster > service > service name (frontend-container-service-4fcey4yx)



Task > click on task link

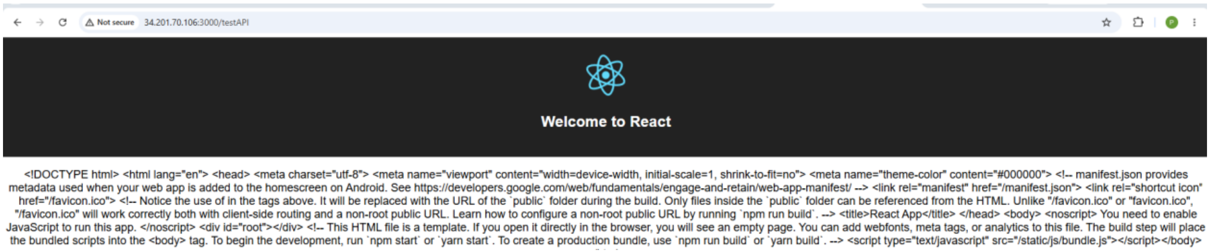


Copy public IP



Web page URL

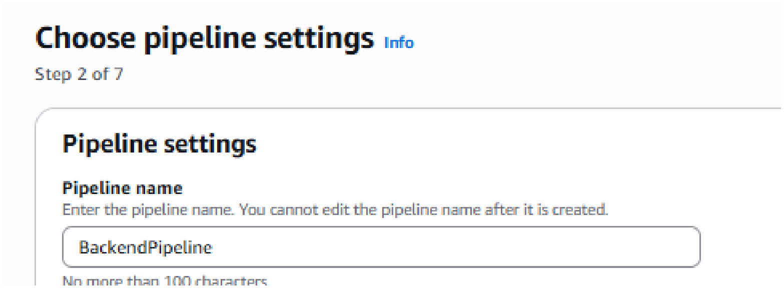
http://34.201.70.106:3000/testAPI

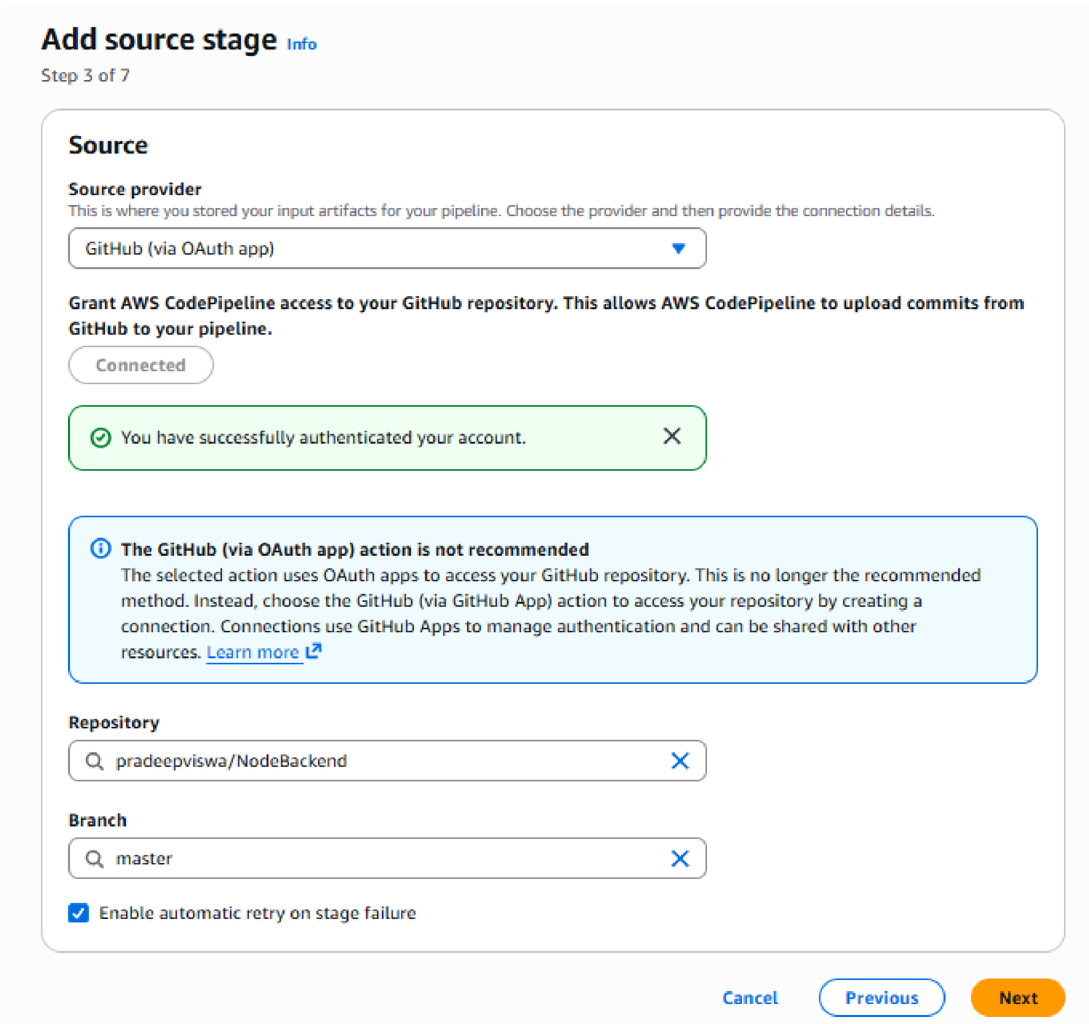


Create Code pipeline for Backend

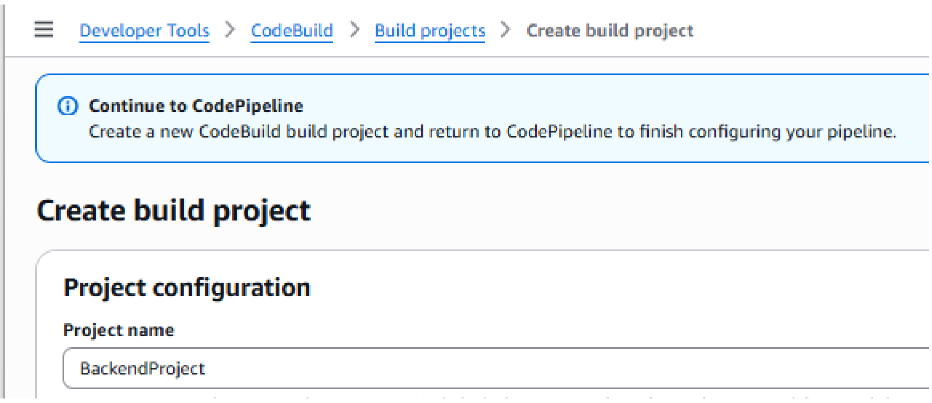
Most of the steps are same as frontendPipeline, only difference screen shots mentioned below

Pipeline name: BackendPipeline

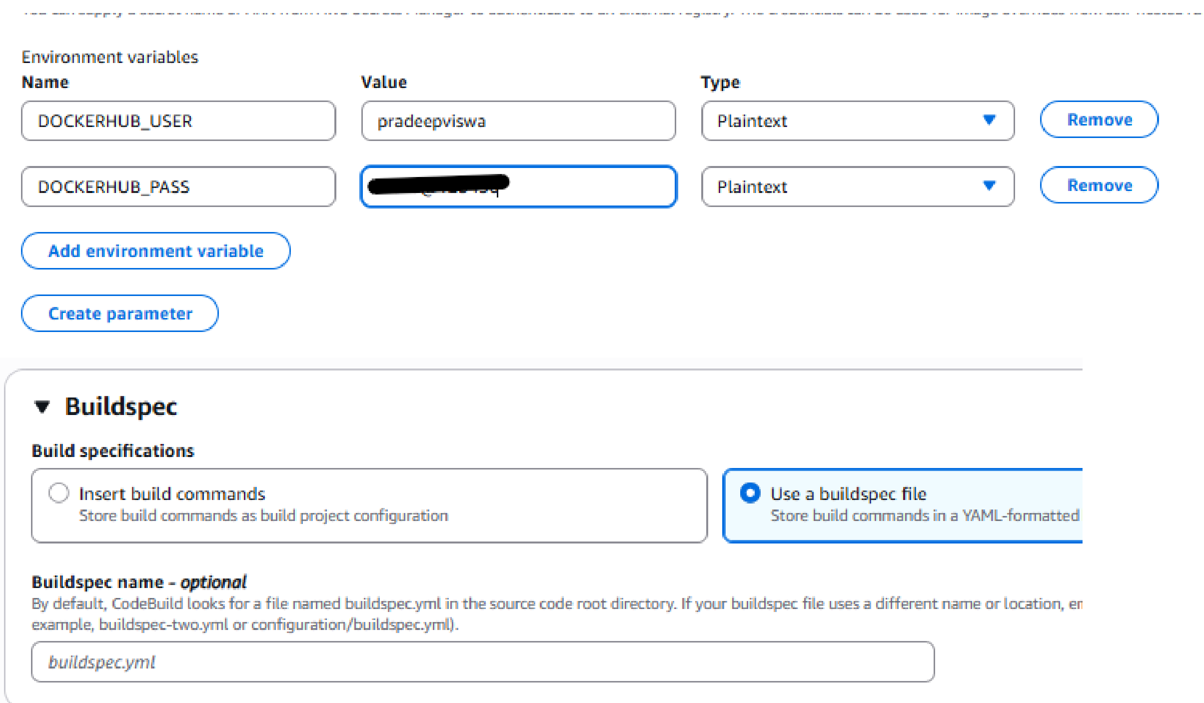


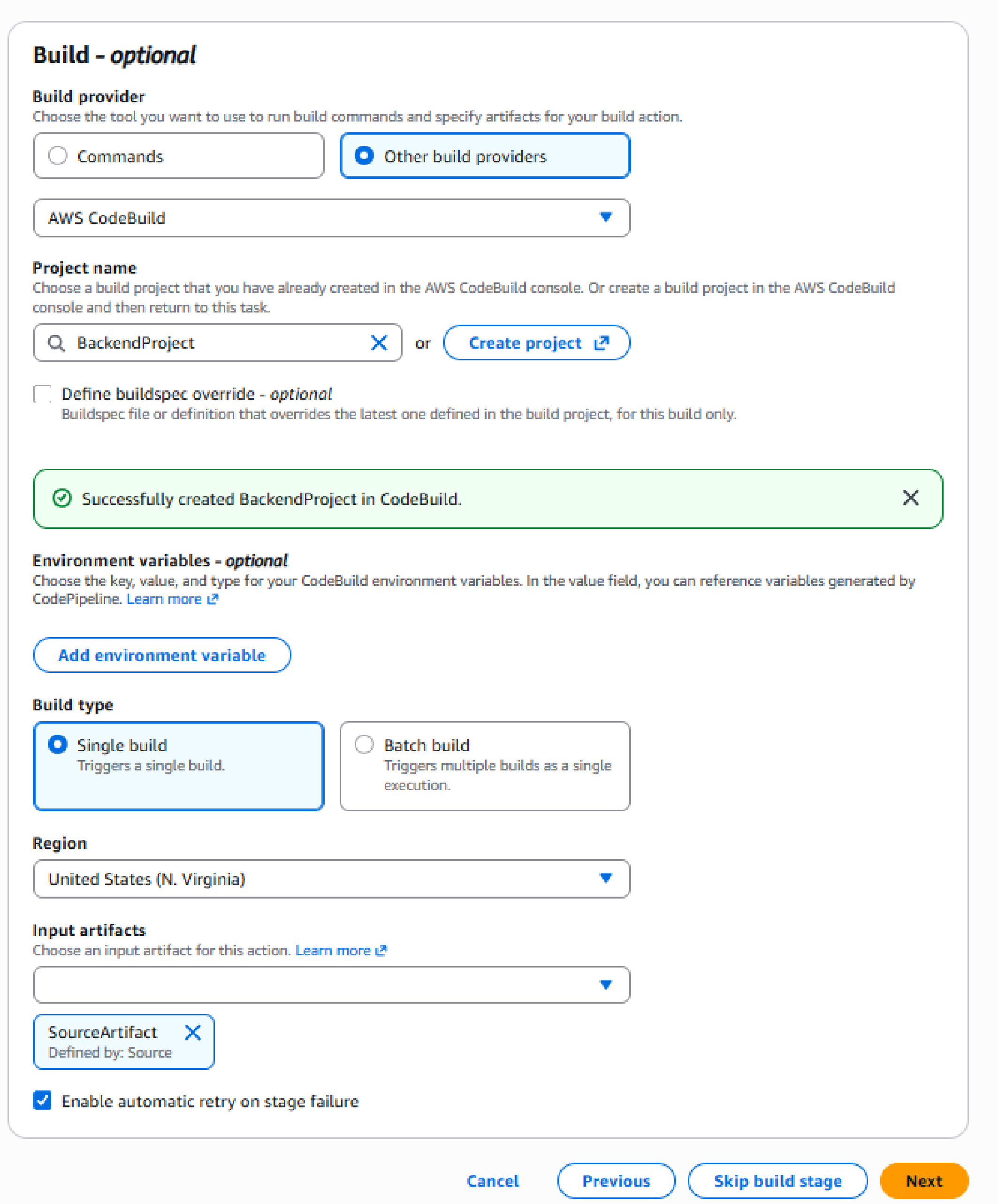


Project Name: BackendProject

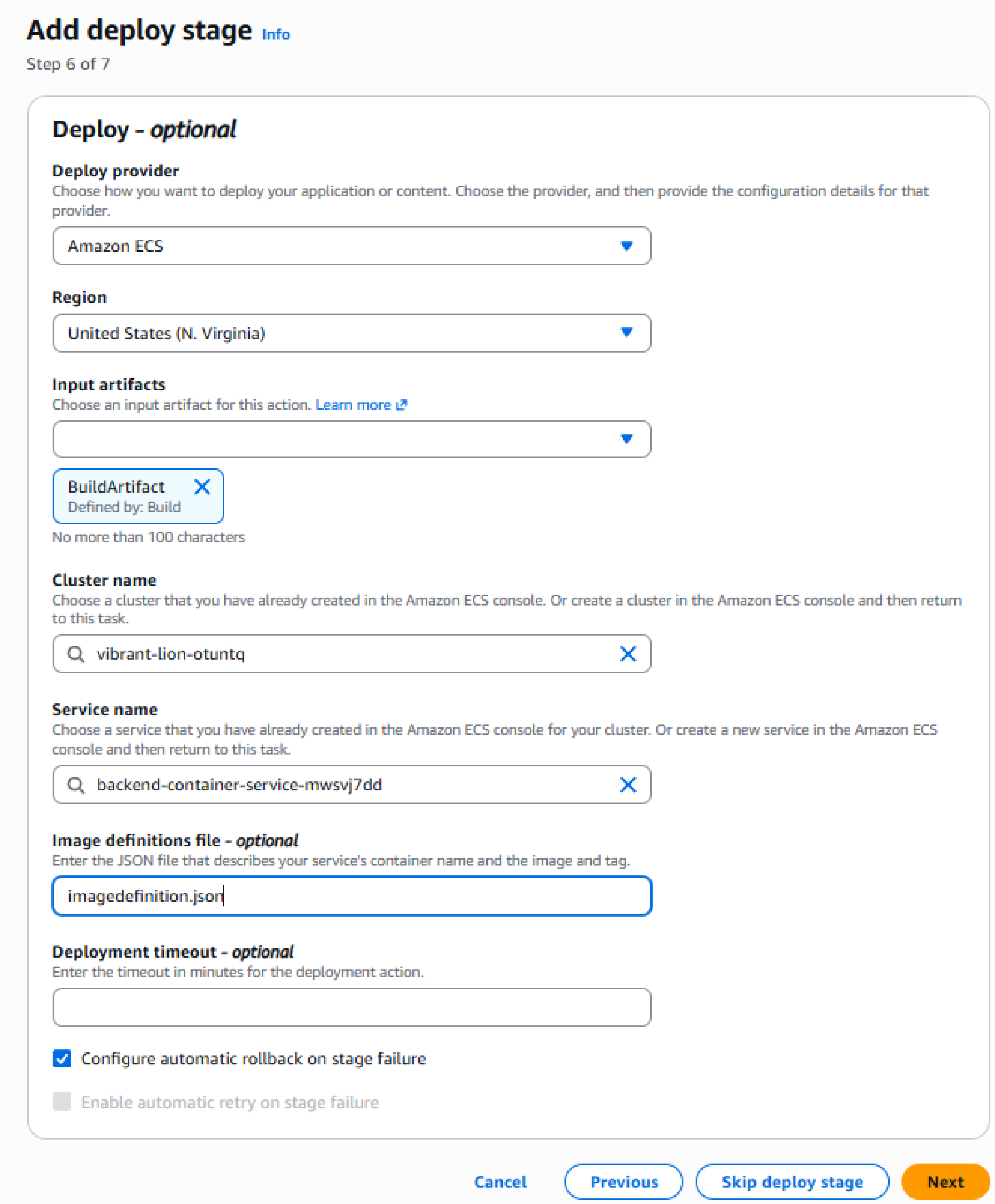


Create: “DOCKERHUB_USER” and “DOCKERHUB_PASS”

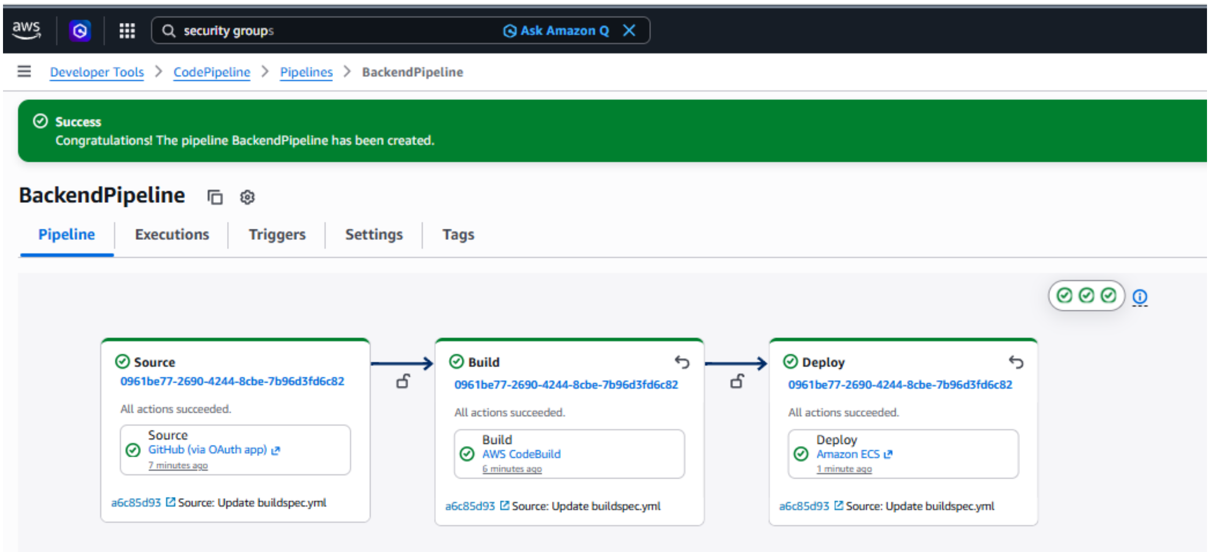



Build - optional
Build provider
Choose the tool you want to use to run build commands and specify artifacts for your build action.
Commands
Other build providers
AWS CodeBuild
Project name
Choose a build project that you have already created in the AWS CodeBuild console. Or create a build project in the AWS CodeBuild console and then return to this task.
BackendProject
or
Create project
Define buildspec override - optional
Buildspec file or definition that overrides the latest one defined in the build project, for this build only.
Successfully created BackendProject in CodeBuild.
Environment variables - optional
Choose the key, value, and type for your CodeBuild environment variables. In the value field, you can reference variables generated by CodePipeline. Learn more
Add environment variable
Build type
Single build
Triggers a single build.
Batch build
Triggers multiple builds as a single execution.
Region
United States (N. Virginia)
Input artifacts
Choose an input artifact for this action. Learn more
SourceArtifact
Defined by: Source
Enable automatic retry on stage failure
Cancel
Previous
Skip build stage
Next

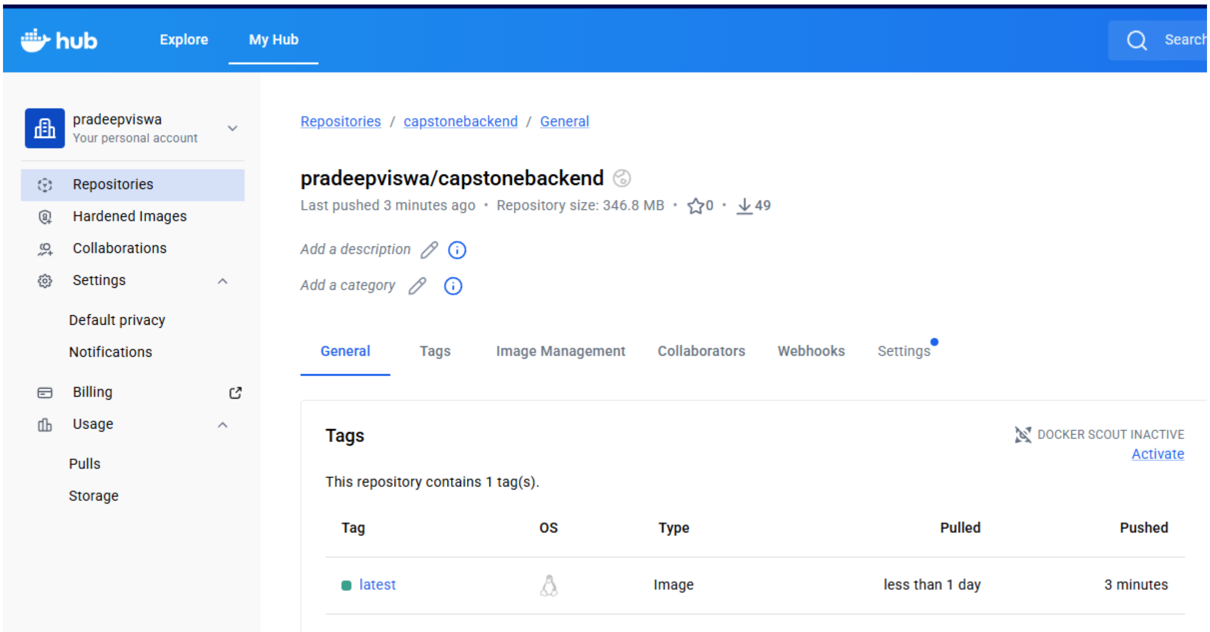
"imagedefinition.json"(for backend pipeline)



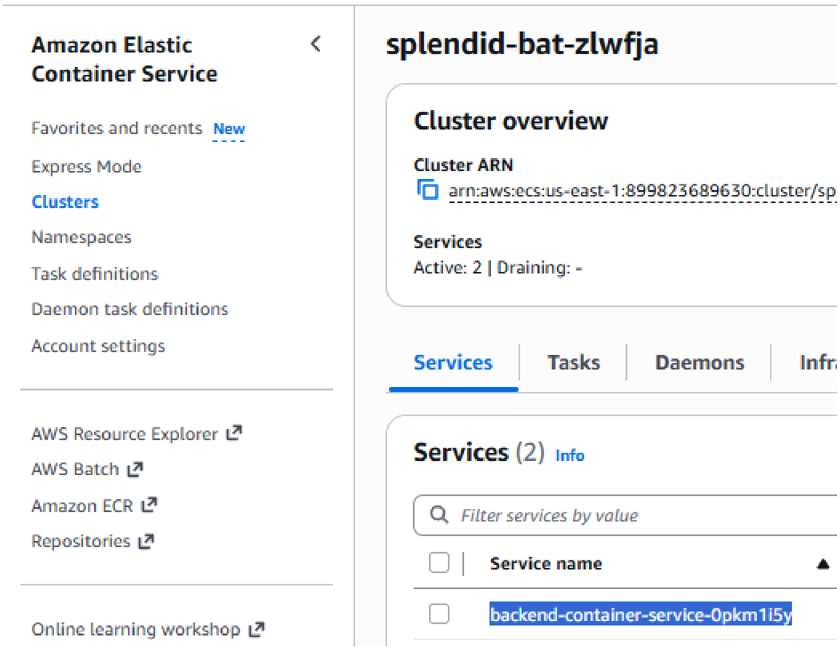
Pipeline

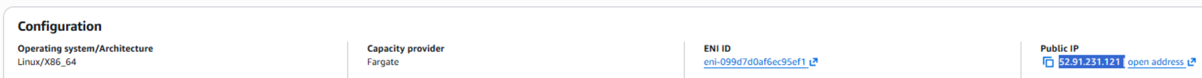


Docker image updated



Get backend public IP from ECS cluster

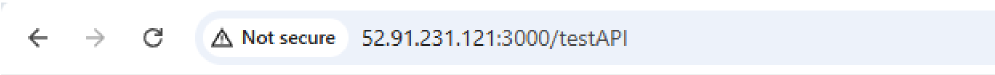




Web Page

IP is 52.91.231.121

URL: http://52.91.231.121:3000/testAPI



API is working properly

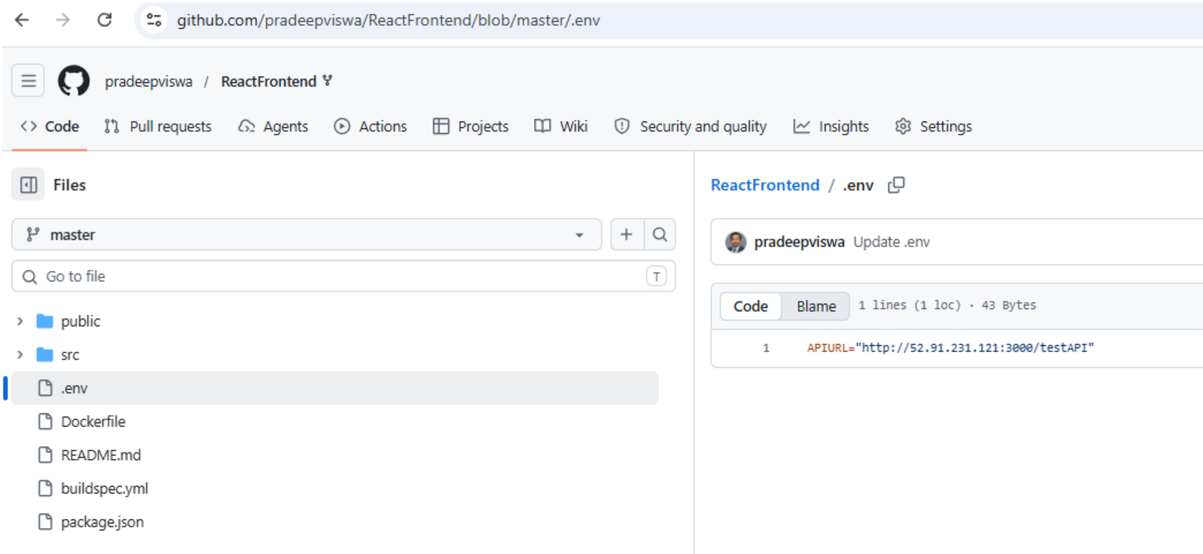
Point Frontend to correct Backend

In frontend source code, we need to modify the .env file where the backend API URL will be updated. get the correct IP address of the backend task, and update the URL

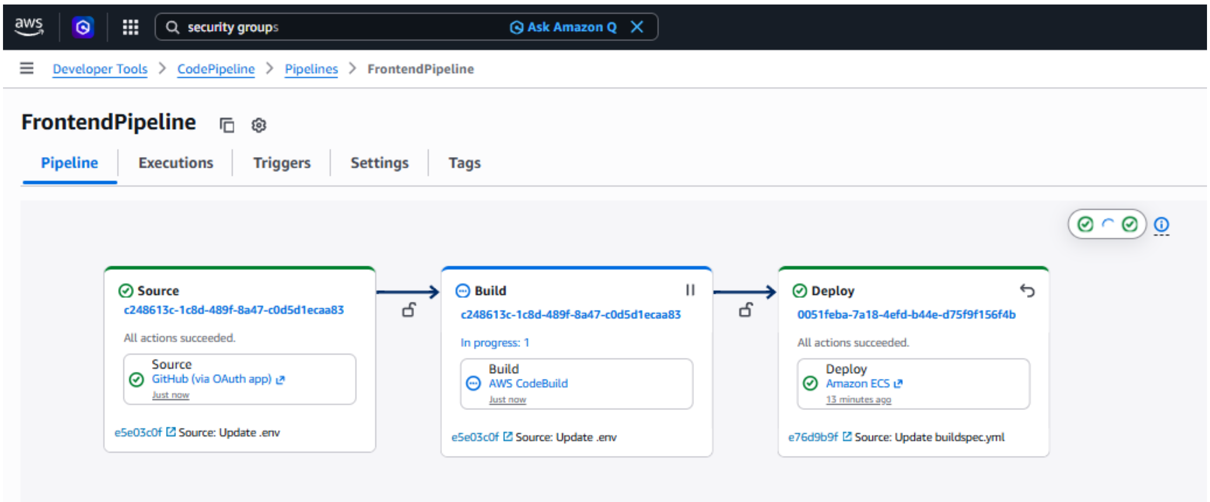
1. File: ReactFrontend/.env
2. file content:

a. APIURL="http://*<backend-IP-Addredd>*:3000/testAPI"

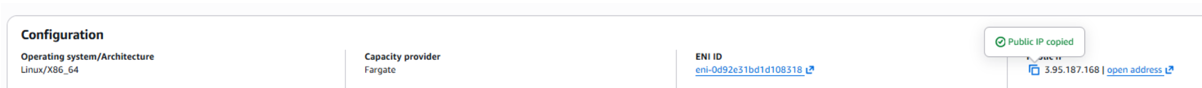
http:// 52.91.231.121:3000/testAPI



After commit build pipeline will trigger again, once done, browse frontend webpage again



After new build, public ip of frontend has changed



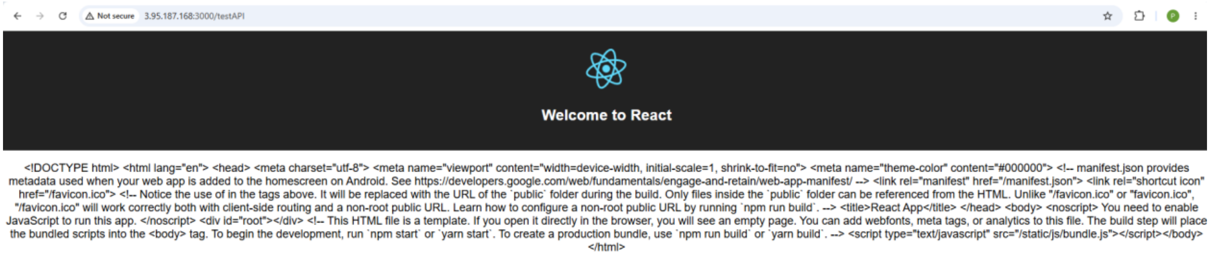
Browse frontend webpage again

URLs:

Frontend: http://3.95.187.168:3000/testAPI

Backend: http://52.91.231.121:3000/testAPI

Browse Frontend again



API communication is failing because of some bug in nodejs code which has to be handled by application developer team. From DevOps point of view, CI/CD pipeline is working as expected.